

Universidad de Costa Rica
Facultad de Ingeniería
Escuela de Ingeniería Eléctrica
IE 0435 – Inteligencia Artificial Aplicada a la Ingeniería Eléctrica
I ciclo 2026

Proyecto 1 - Entrega 3

Danna Guevara Quesada C23562
Grupo 01

Profesor: Marvin Coto Jiménez

Fecha de entrega: 20 de mayo

Índice

1. Conjunto de Datos	1
1.1. Descripción general	1
1.2. Recolección de datos	1
1.3. Preprocesamiento	2
1.4. Conformación del conjunto de datos	3
1.5. Formato del dataset	3
1.6. Limitaciones	3
2. Entrenamiento y selección del modelo	4
2.1. Modelos evaluados	4
2.2. Búsqueda de hiperparámetros	4
2.3. Resultados obtenidos	4
2.4. Hiperparámetros seleccionados	5
2.5. Modelo seleccionado	5
2.6. Matriz de confusión	5
2.7. Exportación del modelo	5
2.8. Generalización y dificultades	5
3. Reproducibilidad del proyecto	6
3.1. Repositorio	7

Índice de figuras

1. Ejemplos de imágenes recolectadas. Izquierda: Imagen con granos de arroz. Derecha: Imagen con clips.	1
2. Ejemplo del proceso de preprocesamiento. Izquierda: Imagen original. Derecha: Imagen binarizada con umbral de 130.	2
3. Ejemplo de un fragmento de la matriz obtenida a partir de una imagen.	3

1. Conjunto de Datos

1.1. Descripción general

El presente conjunto de datos fue construido con el objetivo de entrenar un modelo de clasificación binaria capaz de detectar la presencia de contaminaciones (granos de arroz) en imágenes de una línea de producción simulada. Este problema se enmarca dentro de la inspección visual automatizada en procesos de manufactura.

1.2. Recolección de datos

Siguiendo lo establecido en el planteamiento del proyecto, la línea de producción fue simulada utilizando una hoja de papel blanca como fondo uniforme. Sobre esta superficie se colocaron distintos objetos para generar las muestras.

- **Clase positiva (1):** Imágenes con presencia de granos de arroz.
- **Clase negativa (0):** Imágenes sin granos de arroz, incluyendo:
 - Objetos distintos como clips
- **Total de imágenes:** 30 (15 positivas y 15 negativas)
- **Dispositivo de captura:** Cámara del teléfono
- **Condiciones:** Iluminación de interiores con posibles variaciones de sombras y reflejos

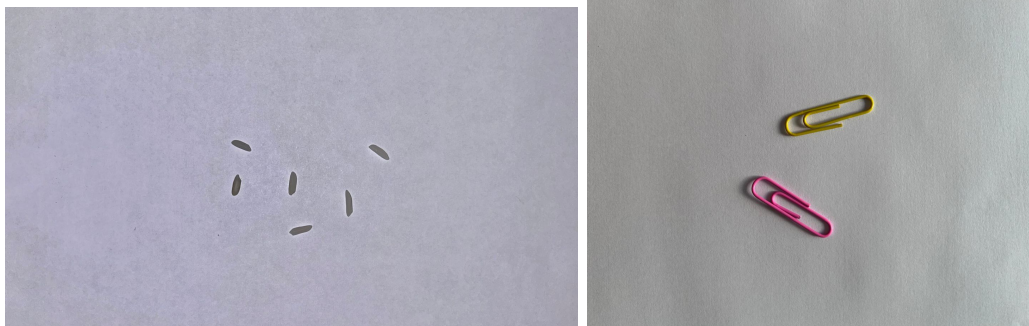


Figura 1: Ejemplos de imágenes recolectadas. Izquierda: Imagen con granos de arroz. Derecha: Imagen con clips.

1.3. Preprocesamiento

Cada imagen fue sometida a un proceso de estandarización con el fin de facilitar su uso en modelos de aprendizaje automático:

1. Conversión a escala de grises
2. Recorte central: se elimina el 20 % de cada borde para reducir ruido de sombras y bordes
3. Redimensionamiento a 128×128 píxeles
4. Binarización mediante un umbral fijo de 130:
 - $\text{Píxel} \geq 130 \rightarrow 1$ (fondo)
 - $\text{Píxel} < 130 \rightarrow 0$ (presencia de objeto)

Este proceso permite representar cada imagen como una matriz binaria de tamaño 128×128 .

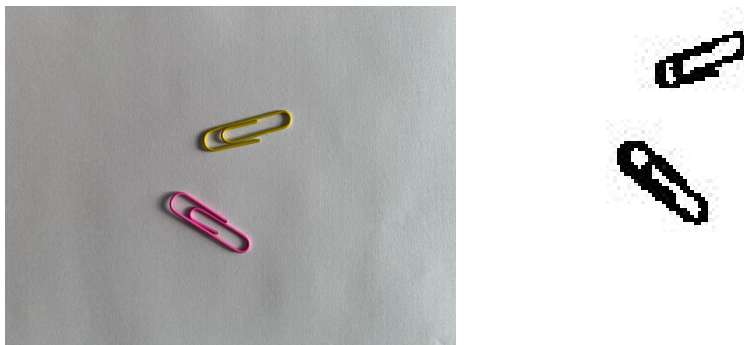


Figura 2: Ejemplo del proceso de preprocesamiento. Izquierda: Imagen original. Derecha: Imagen binarizada con umbral de 130.

1.4. Conformación del conjunto de datos

Siguiendo la metodología indicada en el proyecto, cada imagen binarizada fue transformada en una matriz de valores binarios (0 y 1), donde:

- El valor 1 representa un píxel blanco (fondo)
- El valor 0 representa la presencia de un objeto

Posteriormente, cada matriz fue convertida en un vector fila de 16,384 elementos. A este vector se le añadió una última columna correspondiente a la etiqueta de clase:

- 1: presencia de granos de arroz
- 0: ausencia de granos de arroz

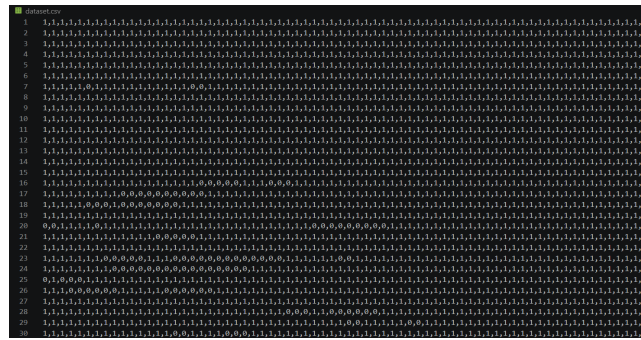


Figura 3: Ejemplo de un fragmento de la matriz obtenida a partir de una imagen.

1.5. Formato del dataset

El conjunto de datos final se almacena en un archivo `dataset.csv` con las siguientes características:

- **Número de filas:** 30
- **Número de columnas:** 16,385
 - 16,384 características (píxeles binarios)
 - 1 etiqueta de clase

1.6. Limitaciones

- La iluminación no es completamente uniforme, lo que puede afectar la binarización, por esto no utilice solo la escala de grises y busque que se implementara el fondo blanco y negro el objeto para que fuera mas seguro.
- El umbral utilizado fue seleccionado empíricamente y podría no generalizar a otros entornos.
- El conjunto de datos es pequeño.
- Objetos pequeños o parcialmente visibles pueden no ser representados adecuadamente.
- Sombras en los bordes pueden introducir ruido en la representación binaria.

2. Entrenamiento y selección del modelo

Para la etapa de entrenamiento se utilizó el dataset combinado del grupo, compuesto por un total de 150 imágenes procesadas. El conjunto de datos se encontraba balanceado, con 75 imágenes positivas y 75 imágenes negativas. Cabe mencionar, que algunos de los archivos utilizados tenían encabezados o formatos ligeramente distintos, por lo que fue necesario adaptar el proceso de lectura y combinación antes del entrenamiento; para este se utilizó una partición estratificada del conjunto de datos, asignando el 80 % de las imágenes al conjunto de entrenamiento y el 20 % al conjunto de prueba. De esta forma, se utilizaron 120 imágenes para entrenar los modelos y 30 imágenes para evaluar su desempeño.

2.1. Modelos evaluados

Se evaluaron cuatro algoritmos clásicos de clasificación supervisada: Árbol de Decisión, Naive Bayes, KNN y SVM.

Estos algoritmos fueron seleccionados porque corresponden a modelos clásicos de clasificación y permiten comparar diferentes enfoques para resolver el problema.

2.2. Búsqueda de hiperparámetros

Para cada algoritmo se realizó una búsqueda de hiperparámetros mediante `GridSearchCV`, utilizando validación cruzada estratificada. Los hiperparámetros son configuraciones definidas antes del entrenamiento que afectan el comportamiento del modelo.

En el caso de SVM, se probaron diferentes valores de `C`, distintos tipos de `kernel` y valores de `gamma`. Para KNN se evaluaron diferentes cantidades de vecinos, métricas de distancia y tipos de ponderación. En Árbol de Decisión se probaron distintos criterios de división, profundidades máximas y cantidades mínimas de muestras para dividir nodos. Para Naive Bayes se evaluaron diferentes valores de suavizado mediante el parámetro `alpha`.

2.3. Resultados obtenidos

En la Tabla 1 se muestran los resultados obtenidos por cada algoritmo evaluado.

Modelo	Accuracy	Precision	Recall	F1-score
Árbol de Decisión	73.33 %	68.42 %	86.67 %	76.47 %
Naive Bayes	50.00 %	50.00 %	13.33 %	21.05 %
KNN	53.33 %	52.00 %	86.67 %	65.00 %
SVM	80.00 %	76.47 %	86.67 %	81.25 %

Tabla 1: Resultados comparativos de los modelos evaluados.

2.4. Hiperparámetros seleccionados

En la Tabla 2 se muestran los mejores hiperparámetros encontrados para cada modelo.

Modelo	Hiperparámetros seleccionados
Árbol de Decisión	<code>criterion=gini, max_depth=10, min_samples_split=5</code>
Naive Bayes	<code>alpha=0.5</code>
KNN	<code>metric=euclidean, n_neighbors=7, weights=distance</code>
SVM	<code>C=10, gamma=scale, kernel=rbf</code>

Tabla 2: Mejores hiperparámetros encontrados por modelo.

2.5. Modelo seleccionado

El mejor algoritmo encontrado fue SVM con kernel RBF, utilizando los hiperparámetros `C=10`, `gamma=scale` y `kernel=rbf`. Este modelo obtuvo el mejor desempeño general, con un accuracy de 80.00 % y un F1-score de 81.25 %.

Se seleccionó el F1-score como métrica principal porque permite balancear la precisión del modelo y su capacidad para detectar correctamente las imágenes positivas. Esto resulta importante porque no solo interesa que el modelo acierte en general, sino que también logre identificar adecuadamente las imágenes con arroz.

2.6. Matriz de confusión

La matriz de confusión obtenida por el modelo SVM fue la siguiente:

$$\begin{bmatrix} 11 & 4 \\ 2 & 13 \end{bmatrix}$$

Esto significa que el modelo clasificó correctamente 11 imágenes negativas y 13 imágenes positivas. Además, clasificó incorrectamente 4 imágenes negativas como positivas y 2 imágenes positivas como negativas.

2.7. Exportación del modelo

El modelo seleccionado fue exportado correctamente utilizando la librería `joblib`, en el archivo `mejor_modelo.pkl`. Además, los resultados de los experimentos fueron almacenados en el archivo `resultados_modelos.json`.

2.8. Generalización y dificultades

El modelo SVM mostró el mejor desempeño en el conjunto de prueba, por lo que se considera el modelo con mejor capacidad de generalización dentro de los experimentos realizados. Sin embargo, el rendimiento puede verse afectado por factores como diferencias de iluminación, fondo, distancia de la cámara, calidad de imagen o variaciones en la forma en que cada integrante capturó sus fotografías.

Una dificultad durante el proceso fue la combinación de los datasets de los compañeros, ya que algunos archivos contenían encabezados o formatos ligeramente diferentes. Por esta razón, fue necesario adaptar el proceso de lectura y combinación para asegurar que todos los archivos tuvieran la misma estructura: 16384 características y una etiqueta final.

A pesar de estas limitaciones, el resultado obtenido se considera aceptable para un conjunto de datos pequeño y variado. Además, el proceso es reproducible gracias al uso de scripts separados para el procesamiento de imágenes, combinación de datasets, entrenamiento, evaluación y exportación del modelo.

3. Reproducibilidad del proyecto

Para asegurar la reproducibilidad del proyecto, se organizó el repositorio de forma que el procesamiento de imágenes, la combinación de datasets, el entrenamiento de modelos y la exportación del modelo final puedan ejecutarse siguiendo una secuencia clara de scripts.

La estructura general del repositorio es la siguiente:

```
ucr-artificial-intelligence/  
|-- fotos_originales/  
|   |-- positivas/  
|   |-- negativas/  
|  
|-- fotos_procesadas/  
|   |-- positivas/  
|   |-- negativas/  
|  
|-- Etiquetas.py  
|-- Datasets.py  
|-- Modelos.py  
|-- dataset.csv  
|-- dataset_grupo.csv  
|-- mejor_modelo.pkl  
|-- resultados_modelos.json  
|-- ProyectoIA_C23562.pdf  
|-- README.md  
|-- MODELCARD.md
```

El archivo `Etiquetas.py` se encarga de procesar las imágenes originales, convertirlas a escala de grises, aplicar recorte central, redimensionarlas a 128x128 píxeles, binarizarlas y generar el archivo `dataset.csv`. Cada fila del dataset contiene 16384 valores correspondientes a los píxeles de la imagen procesada y una etiqueta final.

El archivo `Datasets.py` permite combinar los datasets generados por los integrantes del grupo en un único archivo llamado `dataset_grupo.csv`. Durante este proceso se verificó que las filas tuvieran la cantidad esperada de columnas y se ignoraron encabezados presentes en algunos archivos.

El archivo `Modelos.py` realiza el entrenamiento de los modelos clásicos de clasificación, la búsqueda de hiperparámetros mediante `GridSearchCV`, el cálculo de métricas y la exportación del mejor modelo mediante `joblib`.

Para reproducir el proceso completo, se deben ejecutar los scripts en el siguiente orden:

```
python Etiquetas.py  
python Datasets.py  
python Modelos.py
```

El entrenamiento utiliza una partición estratificada de 80 % para entrenamiento y 20 % para prueba. Además, se empleó `random_state=42` en las partes principales del proceso, lo que permite obtener resultados consistentes entre ejecuciones.

Como resultado final, el proyecto genera los siguientes archivos:

- `dataset.csv`: dataset individual generado a partir de las imágenes propias.
- `dataset_grupo.csv`: dataset combinado del grupo.

- `mejor_modelo.pkl`: modelo final exportado.
- `resultados_modelos.json`: métricas e hiperparámetros obtenidos durante los experimentos.

Esta organización permite que otra persona pueda clonar o descargar el repositorio, ejecutar los scripts en el orden indicado y obtener nuevamente el dataset procesado, los resultados de entrenamiento y el modelo final exportado.

3.1. Repositorio

Repositorio de GitHub: <https://github.com/dguevara8/ucr-artificial-intelligence.git>